

MODULE-I

Short Answer Questions

1. Explain Briefly about Hoisting Process in JavaScript Programming with an Example?

Answer:

Hoisting is JavaScript's default behavior of **moving declarations to the top of their scope before execution**.

This allows some elements to be used before they are explicitly declared in the code, though their behaviour varies significantly based on how they are declared.

2.1 Function Declarations

Hoisting is a JavaScript mechanism where **function declarations** are moved to the top of their scope before code execution.

```
1 let x = 15;
2 let y = 10;
3 let result = add(x, y);
4 console.log(result); // 25
5
6 function add(a, b) {
7   return a + b;
8 }
```

2.2 Function Expressions

Function expressions in JavaScript are not hoisted.

```
1 myFunction();
2 let myFunction = function () {
3   let x = 5;
4   console.log(x); // ReferenceError: "Cannot access 'myFunction' before initialization"
5 };
```

JAVASCRIPT

2.3 Arrow Functions

Arrow Functions in JavaScript are not hoisted.

```
1 myFunction();
2 let myFunction = () => {
3   let x = 5;
4   console.log(x); // ReferenceError: "Cannot access 'myFunction' before initialization"
5 };
```

JAVASCRIPT

2. What is a Variable? Explain its scope of Variable with a simple Example?

Answer:

a) Global Scope

✓ Definition:

Global Scope means a variable is declared **outside of all functions and blocks**. It can be accessed **anywhere in the program**.

It can be accessed anywhere in the program.

✓ Example:

```
javascript Copy code

let name = "Shaik"; // Global variable

function greet() {
  console.log(name); // Accessible here
}

greet();
console.log(name); // Also accessible here
```

b) Local Scope (Function Scope)

✓ Definition:

Local Scope means a variable is declared **inside a function**. It can be accessed **only inside that function**.

✓ Example:

```
javascript Copy code

function greet() {
  let message = "Hello"; // Local variable
  console.log(message);
}

greet();
console.log(message); // ✗ Error (not accessible)
```

c) Block Scope

✓ Definition:

Block Scope means a variable is declared inside a **block { }** using **let** or **const**. It can be accessed **only inside that block**.

✅ **Example:**

```
javascript Copy code  
  
if (true) {  
  let age = 25; // Block scoped variable  
  console.log(age);  
}  
  
console.log(age); // ❌ Error
```

3. What problem does asynchronous JavaScript solve in single-threaded execution?

Answer:

JavaScript is single-threaded, meaning it can do only one task at a time. If a task takes a long time (like loading data from the internet), it can block all other tasks, making the application slow or unresponsive.

Asynchronous JavaScript solves this problem by allowing long tasks to run in the background, so the main program can continue doing other work without waiting.

Example: Imagine you click a button to load data from a server.

=> Without asynchronous behavior, the application will freeze and wait until the data is fully loaded.

=> With asynchronous behavior, the application continues to respond (you can click other buttons or scroll), and the data appears once it is ready.

4. How the 'this' Keyword will process in JavaScript Programming explain with an example?

Answer:

this keyword in JavaScript refers to the **object that is currently executing the function**.

It is a way to access and manipulate the properties and methods of the object that is called the function

```
const person = {  
  name: "Anish",  
  showName() {  
    console.log(`Hello, my name is ${this.name}`);  
  }  
};  
person.showName();
```

output:

Hello, my name is Anish

5. Explain the Types of Destructuring with Syntax in JavaScript Programming with simple Examples?

Destructuring in JavaScript

Destructuring is a way to **extract values from arrays or objects and assign them to variables easily.**

1. Array Destructuring

📌 Used to extract values from arrays.

📄 JavaScript

```
let [var1, var2] = array;
```

Example:

📄 JavaScript

```
let numbers = [10, 20, 30];
```

```
let [a, b] = numbers;
```

```
console.log(a); // 10
```

```
console.log(b); // 20
```

2. Object Destructuring

📖 Used to extract values from objects.

Syntax:

```
</> JavaScript
let {key1, key2} = object;
```

Example:

```
</> JavaScript
let person = {
  name: "Abbas",
  age: 21
};

let {name, age} = person;

console.log(name); // Abbas
console.log(age); // 21
```



3. Default Values

📖 Assign default values if data is missing.

Example:

```
</> JavaScript
let [a = 5, b = 10] = [1];

console.log(a); // 1
console.log(b); // 10
```

4. Nested Destructuring

📖 Extract values from nested arrays/objects.

Example:

```
</> JavaScript
let student = {
  name: "Abbas",
  marks: {
    math: 90
  }
};

let {marks: {math}} = student;

console.log(math); // 90
```

Destructuring makes code **short, readable, and easy to use** when working with arrays and objects.

6. Explain briefly about Object Destructuring in JavaScript Programming with an Example?

Answer:

✓ Definition:

Object Destructuring is a JavaScript feature that allows us to **extract properties from an object and store them into variables easily**.

👉 Instead of accessing values using `object.property`, we can directly unpack them into variables.

📌 Quick Syntax

javascript

📄 Copy code

```
const { property1, property2 } = objectName;
```

◆ Normal Way (Without Destructuring)

javascript

📄 Copy code

```
const person = {
  name: "Shaik",
  age: 25,
  city: "Hyderabad"
};

const name = person.name;
const age = person.age;

console.log(name);
console.log(age);
```

◆ Using Object Destructuring

javascript

Copy code

```
const person = {  
  name: "Shaik",  
  age: 25,  
  city: "Hyderabad"  
};  
  
const { name, age } = person;  
  
console.log(name);  
console.log(age);
```

7. What is the Syntax for Arrow Function in JavaScript Programming, Explain with an example?

✓ Definition:

An Arrow Function is a shorter and modern way to write functions in JavaScript using the => (arrow) syntax.

👉 Introduced in ES6 (ES2015)

👉 Makes code shorter and cleaner

Syntax:

```
let sum = (param1, param2, ...) => {  
  // statement(s)  
};  
sum();
```

```
1 let sum = (a, b) => {  
2   let result = a + b;  
3   return result;  
4 };  
5 console.log(sum(4, 3));
```

JAVASCRIPT

Long Answer Questions

1. Explain Default binding and Implicit binding of “this” keyword along with example program?

Default Binding and Implicit Binding of this in JavaScript

In JavaScript, the value of **this** keyword depends on **how a function is called**, not where it is defined. Two important types of binding are **Default Binding** and **Implicit Binding**.

Default Binding

Definition:

Default binding occurs when a regular function is called **without any object reference**

1.2 `this` in Regular Functions

```
1 function start() {  
2   console.log(this); // Window { }  
3 }  
4 start();
```

In the above example, `this` refers to the `window` object.

Implicit Binding

Definition:

Implicit binding occurs when a function is called **as a method of an object**. In this case:

- `this` refers to the **object that is calling the function**.

1.1 `this` in Object Methods

```
1 let car = {  
2   color: "blue",  
3   brand: "Audi",  
4   start: function() {  
5     console.log(this); // Object { color: "blue", brand: "Audi", start: f() }  
6   }  
7 };  
8  
9 car.start();
```

In the above example, `this` refers to the `car` object as it's executing the method `start`.

2. How is Error Handling will handle in JavaScript Programing? Explain the Error handling Mechanisms?

Answer: -

Error Handling in JavaScript is the process of detecting, managing, and responding to runtime errors so that the program does not crash unexpectedly.

JavaScript provides built-in mechanisms to handle errors gracefully and maintain smooth execution.

Types of Errors in JavaScript

1. Syntax Error

Occurs when code violates JavaScript rules.

javascript

 Copy code

```
console.log("Hello"
```

(Missing closing bracket)

2. Runtime Error

Occurs during execution.

javascript

 Copy code

```
let x = y + 5; // y is not defined
```

3. Logical Error

Program runs but gives incorrect output.

Error Handling Mechanisms in JavaScript

try...catch...finally

This is the main error handling mechanism.

Syntax:

javascript

 Copy code

```
try {  
    // Code that may cause error  
}  
catch(error) {  
    // Code to handle error  
}  
finally {  
    // Code that always executes  
}
```

try Block

The try block contains the code that may generate an error.

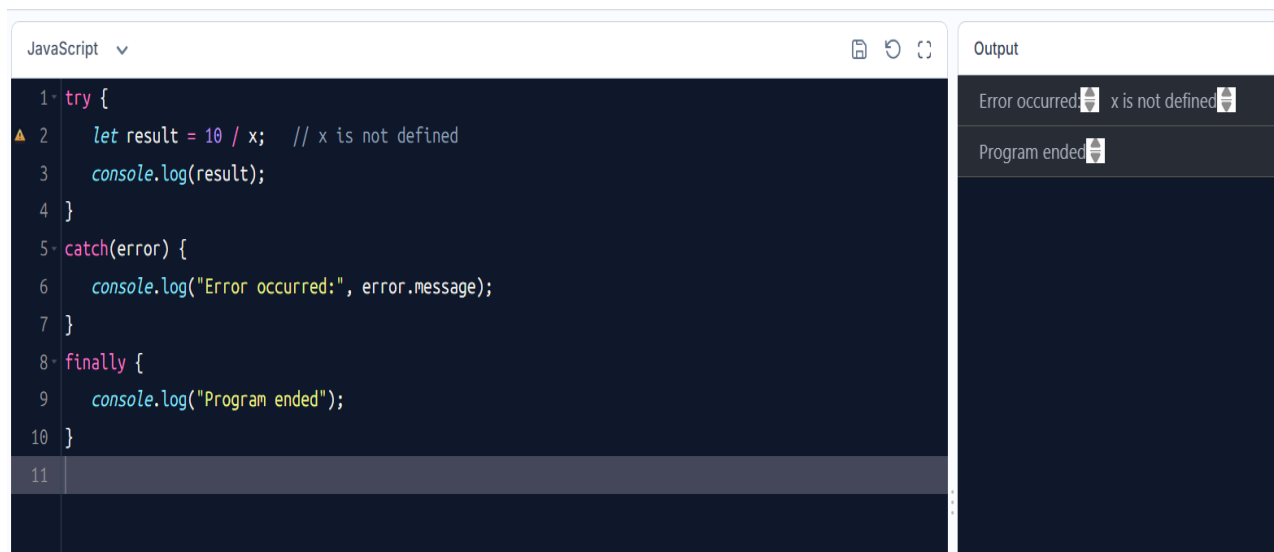
catch Block

The catch block handles the error.

finally Block

The finally block always executes, whether an error occurs or not.

Example: -



The screenshot shows a code editor with the following JavaScript code:

```
1 try {  
2   let result = 10 / x; // x is not defined  
3   console.log(result);  
4 }  
5 catch(error) {  
6   console.log("Error occurred:", error.message);  
7 }  
8 finally {  
9   console.log("Program ended");  
10 }  
11
```

The output panel on the right shows the following messages:

```
Output  
Error occurred: x is not defined  
Program ended
```

3. Explain briefly about Classes mechanism in JavaScript Programming in detail along with example?

Answer: -

Classes in JavaScript are a blueprint for creating objects. They were introduced in **ECMAScript 6 (ES6)** in 2015.

Classes provide a cleaner and more structured way to create objects and handle inheritance compared to traditional constructor functions.

1.1. Constructor Method

The constructor method is a special method of a class for creating and initializing an object of that class.

Syntax:

```
class MyClass {  
  constructor(property1, property2) {  
    this.property1 = property1;  
    this.property2 = property2;  
  }  
  method1() { ... }  
  method2() { ... }  
}
```

Example: -

```
class Student {  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
  
  greet() {  
    console.log("Hello, my name is " + this.name);  
  }  
}  
  
let s1 = new Student("Rahul", 20);  
console.log(s1)  
console.log(s1.name)  
console.log(s1.greet())
```

Output

```
Object { name: "Rahul", age: 20 }
```

```
Rahul
```

```
Hello, my name is Rahul
```

Inheritance (extends keyword)

One class can inherit properties and methods from another class.

```

1 class Person {
2     constructor(name) {
3         this.name = name;
4     }
5     greet() {
6         console.log("Hello " + this.name);
7     }
8 }
9 class Employee extends Person {
10     constructor(name, salary) {
11         super(name); // Call parent constructor
12         this.salary = salary;
13     }
14     showSalary() {
15         console.log("Salary: " + this.salary);
16     }
17 }
18 let emp1 = new Employee("Amit", 50000);
19 emp1.greet();
20 emp1.showSalary();
21

```

Output

Hello Amit

Salary: 50000

4. Write the Syntax and Program of any ten Array Methods in JavaScript Programming?

✓ 1. `push()`

► Definition:

Adds one or more elements to the **end** of an array.

► Syntax:

javascript

 Copy code

```
array.push(element1, element2, ...)
```

► Example:

javascript

 Copy code

```
let arr = [1, 2, 3];  
arr.push(4);  
console.log(arr); // [1, 2, 3, 4]
```



✓ 2. `pop()`

► Definition:

Removes the **last element** from an array.

► Syntax:

javascript

 Copy code

```
array.pop()
```

► Example:

javascript

 Copy code

```
let arr = [1, 2, 3];  
arr.pop();  
console.log(arr); // [1, 2]
```



✓ 3. `shift()`

► Definition:

Removes the **first element** from an array.

► Syntax:

javascript

 Copy code

```
array.shift()
```

► Example:

javascript

 Copy code

```
let arr = [10, 20, 30];  
arr.shift();  
console.log(arr); // [20, 30]
```



✓ 4. `unshift()`

► Definition:

Adds one or more elements to the **beginning** of an array.

► Syntax:

javascript

 Copy code

```
array.unshift(element1, element2, ...)
```

► Example:

javascript

 Copy code

```
let arr = [2, 3];  
arr.unshift(1);  
console.log(arr); // [1, 2, 3]
```



✓ 5. `map()`

► Definition:

Creates a **new array** by applying a function to each element.

► Syntax:

javascript

 Copy code

```
array.map(function(element, index, array))
```

► Example:

javascript

 Copy code

```
let arr = [1, 2, 3];  
let doubled = arr.map(num => num * 2);  
console.log(doubled); // [2, 4, 6]
```



✓ 6. `filter()`

► Definition:

Creates a **new array** with elements that pass a condition.

► Syntax:

javascript

 Copy code

```
array.filter(function(element))
```

► Example:

javascript

 Copy code

```
let arr = [1, 2, 3, 4];  
let even = arr.filter(num => num % 2 === 0);  
console.log(even); // [2, 4]
```



✓ 7. `reduce()`

► Definition:

Reduces array to a **single value**.

► Syntax:

javascript

 Copy code

```
array.reduce(function(accumulator, currentValue), initialValue)
```

► Example:

javascript

 Copy code

```
let arr = [1, 2, 3, 4];  
let sum = arr.reduce((total, num) => total + num, 0);  
console.log(sum); // 10
```



✓ 8. `forEach()`

► Definition:

Executes a function for each array element.

► Syntax:

javascript

 Copy code

```
array.forEach(function(element, index))
```

► Example:

javascript

 Copy code

```
let arr = [1, 2, 3];  
arr.forEach(num => console.log(num));
```



✓ 9. slice()

► Definition:

Returns a **new array** by extracting a portion of the array.

► Syntax:

javascript

Copy code

```
array.slice(start, end)
```

► Example:

javascript

Copy code

```
let arr = [10, 20, 30, 40];  
let part = arr.slice(1, 3);  
console.log(part); // [20, 30]
```



✓ 10. length

► Definition:

Returns the **number of elements** in an array.

It can also be used to **modify the size** of an array.

► Syntax:

javascript

Copy code

```
array.length
```

► Example 1: Getting length

javascript

Copy code

```
let arr = [10, 20, 30, 40];  
console.log(arr.length); // 4
```



5. Write the Syntax and Program of any ten String Methods in JavaScript Programming?

✓ 1. `length`

► Definition:

Returns the number of characters in a string.

► Syntax:

```
javascript
```

[Copy code](#)

```
string.length
```

► Example:

```
javascript
```

[Copy code](#)

```
let str = "JavaScript";  
console.log(str.length); // 10
```



✓ 2. toUpperCase()

► Definition:

Converts all characters in a string to **uppercase**.

► Syntax:

javascript

 Copy code

```
string.toUpperCase()
```

► Example:

javascript

 Copy code

```
let str = "hello";  
console.log(str.toUpperCase()); // HELLO
```

✓ 3. toLowerCase()

► Definition:

Converts all characters in a string to **lowercase**.

► Syntax:

javascript

 Copy code

```
string.toLowerCase()
```

► Example:

javascript

 Copy code

```
let str = "HELLO";  
console.log(str.toLowerCase()); // hello
```

✓ 4. trim()

► Definition:

Removes whitespace from both ends of a string.

► Syntax:

javascript

 Copy code

```
string.trim()
```

► Example:

javascript

 Copy code

```
let str = " Hello World ";  
console.log(str.trim()); // "Hello World"
```



✓ 5. charAt()

► Definition:

Returns the character at a specified index.

► Syntax:

javascript

 Copy code

```
string.charAt(index)
```

► Example:

javascript

 Copy code

```
let str = "Java";  
console.log(str.charAt(2)); // v
```



✓ 6. indexOf()

► Definition:

Returns the position of the **first occurrence** of a specified value.

► Syntax:

javascript

 Copy code

```
string.indexOf(searchValue)
```

► Example:

javascript

 Copy code

```
let str = "Hello World";  
console.log(str.indexOf("World")); // 6
```

✓ 7. lastIndexOf()

► Definition:

Returns the position of the **last occurrence** of a specified value.

► Syntax:

javascript

 Copy code

```
string.lastIndexOf(searchValue)
```

► Example:

javascript

 Copy code

```
let str = "hello hello";  
console.log(str.lastIndexOf("hello")); // 6
```

✓ 8. slice()

► Definition:

Extracts a part of a string and returns a new string.

► Syntax:

javascript

Copy code

```
string.slice(start, end)
```

► Example:

javascript

Copy code

```
let str = "JavaScript";  
console.log(str.slice(0, 4)); // Java
```

✓ 9. includes()

► Definition:

Checks whether a string contains a specified value.

► Syntax:

javascript

Copy code

```
string.includes(searchValue)
```

► Example:

javascript

Copy code

```
let str = "I love JavaScript";  
console.log(str.includes("Java")); // ↓ ?
```

✓ 10. concat()

► Definition:

Joins two or more strings.

► Syntax:

javascript

Copy code

```
string.concat(string1, string2, ...)
```

► Example:

javascript

Copy code

```
let str1 = "Hello";  
let str2 = "World";  
console.log(str1.concat(" ", str2));
```



ello World

6. Explain in detail about try block, catch block and finally block in JavaScript Programming along with example program?

Error Handling in JavaScript is the process of detecting, managing, and responding to runtime errors so that the program does not crash unexpectedly.

JavaScript provides built-in mechanisms to handle errors gracefully and maintain smooth execution.

Types of Errors in JavaScript

1. Syntax Error

Occurs when code violates JavaScript rules.

javascript

Copy code

```
console.log("Hello"
```

(Missing closing bracket)

2. Runtime Error

Occurs during execution.

javascript

 Copy code

```
let x = y + 5; // y is not defined
```

3. Logical Error

Program runs but gives incorrect output.

Error Handling Mechanisms in JavaScript

try...catch...finally

This is the main error handling mechanism.

Syntax:

javascript

 Copy code

```
try {  
    // Code that may cause error  
}  
catch(error) {  
    // Code to handle error  
}  
finally {  
    // Code that always executes  
}
```

try Block

The try block contains the code that may generate an error.

catch Block

The catch block handles the error.

finally Block

The finally block always executes, whether an error occurs or not.

Example: -

JavaScript



```
1 try {  
2   let result = 10 / x; // x is not defined  
3   console.log(result);  
4 }  
5 catch(error) {  
6   console.log("Error occurred:", error.message);  
7 }  
8 finally {  
9   console.log("Program ended");  
10 }  
11
```

Output

Error occurred: x is not defined

Program ended

MODULE-II

Short Answer Questions

1. What problem does asynchronous JavaScript solve in single-threaded execution?

Problem in Single-Threaded JavaScript

JavaScript is **single-threaded**, which means it can do **only one task at a time**.

👉 If one task takes a long time (like fetching data from a server), it will **block the execution** of other tasks.

Example Problem:

```
</> JavaScript
console.log("Start");

let data = fetchData(); // takes 5 seconds

console.log("End");
```

✗ Output (blocking):

```
Start
(wait 5 seconds)
End
```

👉 Here, the program **waits** until the long task finishes → this is called **blocking**.

Solution: Asynchronous JavaScript

Asynchronous JavaScript allows **non-blocking execution**.

👉 Long tasks (like API calls, timers) are handled in the background, so other code can run without waiting.

Example (Async):

```
</> JavaScript
console.log("Start");

setTimeout(() => {
  console.log("Data received");
}, 5000);

console.log("End");
```

✓ Output (non-blocking):

```
Start
End
(after 5 seconds) Data received
```

2. How Synchronous programming will process in JavaScript code Explain in detail ?

Answer:

✓ Definition:

Synchronous programming means the code executes **line by line**, one after another.

🔗 JavaScript waits for one task to finish before moving to the next task.

◆ Example:

```
javascript
console.log("Step 1");
console.log("Step 2");
console.log("Step 3");
```

✓ Output:

```
vbnet
Step 1
Step 2
Step 3
```

3. Explain the Promise states in JavaScript Programming of following

a) Pending state

b) Fulfilled state

c) Reject state

Answer:

a) Pending State

✔ Definition:

A Promise is in the **Pending** state when it is **still executing** and the result is not yet available.

👉 Initial state of every Promise.

✔ Example:

javascript

Copy code

```
const promise = new Promise((resolve, reject) => {  
  // Task is running  
});
```

At this stage:

- Not resolved
- Not rejected
- Waiting for result

b) Fulfilled State

✔ Definition:

A Promise is in the **Fulfilled** state when the operation is **successful** and `resolve()` is called.

👉 It returns a value.

✔ Example:

javascript

Copy code

```
const promise = new Promise((resolve, reject) => {  
  resolve("Success");  
});  
  
promise.then((result) => {  
  console.log(result); // Success  
});
```

- ✔ Task completed successfully
- ✔ Value is available

c) Rejected State

✔ Definition:

A Promise is in the **Rejected** state when the operation **fails** and `reject()` is called.

🔑 It returns an error.

✅ **Example:**

```
javascript Copy code

const promise = new Promise((resolve, reject) => {
  reject("Error occurred");
});

promise.catch((error) => {
  console.log(error); // Error occurred
});
```

- ✔ Task failed
- ✔ Error is returned

4. Explain the methods that can resolve in promise of JavaScript Programming?

Answer:

.then(): `.then()` is used to handle a fulfilled (resolved) Promise. Runs only when `resolve()` is called.

Syntax: `promise.then(successCallback);`

.catch(): `.catch()` is used to handle a rejected Promise. Runs only when `reject()` is called.

✅ **Example:**

```
javascript Copy code

const promise = new Promise((resolve, reject) => {
  resolve("Success");
});

promise.then((result) => {
  console.log(result); // Success
});
```

- ✔ Task completed successfully
- ✔ Value is available

✅ **Example:**

```
javascript Copy code
const promise = new Promise((resolve, reject) => {
  reject("Error occurred");
});

promise.catch((error) => {
  console.log(error); // Error occurred
});
```

- ✔ Task failed
- ✔ Error is returned

5. Explain briefly about states in promise Asynchronous operation in JavaScript Programming?

Answer:

a) Pending State

✅ **Definition:**

A Promise is in the **Pending** state when it is **still executing** and the result is not yet available.

👉 Initial state of every Promise.

✅ **Example:**

```
javascript Copy code
const promise = new Promise((resolve, reject) => {
  // Task is running
});
```

At this stage:

- Not resolved
- Not rejected
- Waiting for result

b) Fulfilled State

✅ **Definition:**

A Promise is in the **Fulfilled** state when the operation is **successful** and `resolve()` is called.

👉 It returns a value.

✓ Example:

javascript

Copy code

```
const promise = new Promise((resolve, reject) => {
  resolve("Success");
});

promise.then((result) => {
  console.log(result); // Success
});
```

✓ Task completed successfully

✓ Value is available

c) Rejected State

✓ Definition:

A Promise is in the **Rejected** state when the operation **fails** and `reject()` is called.

☞ It returns an error.

✓ Example:

javascript

Copy code

```
const promise = new Promise((resolve, reject) => {
  reject("Error occurred");
});

promise.catch((error) => {
  console.log(error); // Error occurred
});
```

✓ Task failed

✓ Error is returned

6. How Asynchronous programming will process in JavaScript code Explain in detail ?

Asynchronous programming means JavaScript can **do multiple tasks without waiting** for one task to finish.

☞ It helps to **avoid blocking** and keeps the application fast.

Why We Need It?

If a task takes time (like fetching data from server), JavaScript will:

- ✗ In synchronous → wait and stop everything
 - ✓ In asynchronous → continue other work and handle result later
-

How It Works (Simple)

1. Normal code runs first
2. Long tasks go to background (browser handles it)
3. When task is done → result comes back
4. Then JavaScript executes it

Types of Asynchronous Programming

1. Callbacks

👉 A function is given to another function and runs later.

```
<> JavaScript
setTimeout(() => {
  console.log("Hello");
}, 2000);
```

✓ Runs after 2 seconds

2. Promises

👉 Promise gives result in future.

```
<> JavaScript
let p = new Promise((resolve) => {
  resolve("Done");
});

p.then((res) => console.log(res));
```

✓ More clean than callbacks

3. Async / Await

👉 Makes async code look simple like normal code.

```
<> JavaScript
async function show() {
  let data = await p;
  console.log(data);
}

show();
```

✓ Easy to read and understand



Long Answer Questions

1. What is a Callback Function, and how does it enable asynchronous behaviour in JavaScript explain in detail with an Example?

What is a Callback Function?

✍ A **callback function** is a function that is **passed as an argument to another function**

✍ It is **executed later**, after a task is completed

Why Callbacks are Used?

✍ They help JavaScript handle **asynchronous operations**

✓ Instead of waiting for a task to finish

✓ JavaScript continues execution

✓ When the task is done → callback runs

How Callback Enables Asynchronous Behaviour

1. A long task (like API call / timer) is started
2. JavaScript does **not wait**
3. Task runs in background
4. When task finishes → callback function is called
5. Result is processed

✍ This avoids **blocking** and keeps program fast

Example: Callback Function

```
<> JavaScript   
  
function greet(name, callback) {  
  console.log("Hello " + name);  
  callback();  
}  
  
function sayBye() {  
  console.log("Goodbye!");  
}  
  
greet("Raju", sayBye);
```

Output:

```
Hello Raju  
Goodbye! 
```



Asynchronous Example (with Raju)

```
<> JavaScript   
  
console.log("Raju started task");  
  
setTimeout(() => {  
  console.log("Raju completed task");  
}, 2000);  
  
console.log("Raju is doing other work");
```

Output:

```
Raju started task  
Raju is doing other work  
Raju completed task 
```

Conclusion

 Callback function:

- Executes **after task finishes**
- Helps in **asynchronous (non-blocking)** execution
- Makes JavaScript programs **fast and responsive**

2. Demonstrate how to implement time delays in JavaScript programming with an example?

Timer Functions in JavaScript

Timer functions are used to execute code after a specific delay or repeatedly at fixed time intervals. JavaScript provides two main timer functions:

1. `setTimeout()`
2. `setInterval()`

1) `setTimeout()` Function

► Definition:

`setTimeout()` is used to execute a function **once** after a specified time delay (in milliseconds).

► Syntax:

javascript

 Copy code

```
setTimeout(function, delay, parameter1, parameter2, ...);
```

- `function` → The function to execute
- `delay` → Time in milliseconds (1000 ms = 1 second)

► Example Program:

javascript

 Copy code

```
function greet() {  
    console.log("Hello after 3 seconds!");  
}  
  
setTimeout(greet, 3000);
```

► Output:

After 3 seconds, it prints:

nginx

 Copy code

```
Hello after 3 seconds!
```

2) setInterval() Function

► Definition:

setInterval() is used to execute a function **repeatedly** at specified time intervals.

► Syntax:

javascript

 Copy code

```
setInterval(function, interval);
```

- `function` → The function to execute
- `interval` → Time in milliseconds

► Example Program:

javascript

 Copy code

```
function showTime() {  
    console.log("Running every 2 seconds");  
}  
  
setInterval(showTime, 2000);
```

► Output:

It prints every 2 seconds continuously:

sql

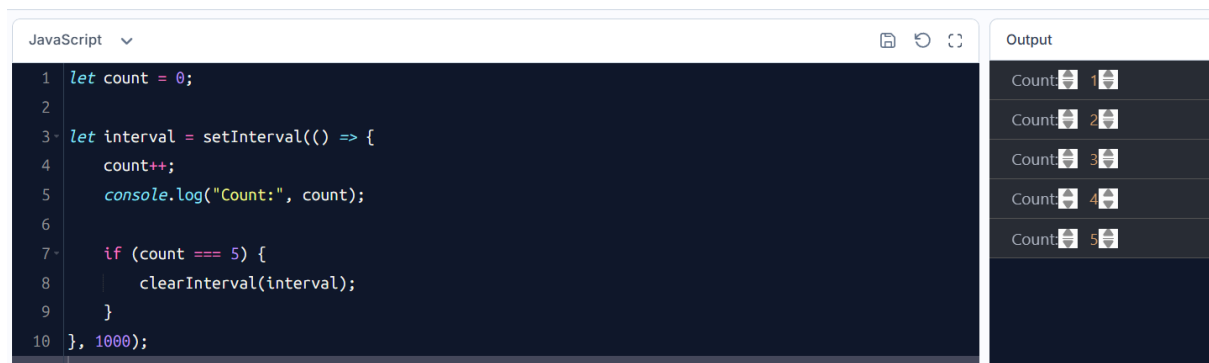
 Copy code

```
Running every 2 seconds  
Running every 2 seconds  
Running every 2 seconds  
...
```



Note: Since the setInterval() method executes the function infinitely hence there is a method called as clearInterval() to stop the execution of the setInterval().

Example using clearInterval() to stop the execution of the setInterval().



```
1 let count = 0;
2
3 let interval = setInterval(() => {
4   count++;
5   console.log("Count:", count);
6
7   if (count === 5) {
8     clearInterval(interval);
9   }
10 }, 1000);
```

Output

Count 1

Count 2

Count 3

Count 4

Count 5

3. How does overusing callbacks lead to unmaintainable code, and what are the solutions Explain in detail?

Answer:

Callback Hell is a situation where multiple asynchronous operations are nested inside one another using callbacks, resulting in deeply indented and hard-to-read code.

It is also called:

- ◆ Pyramid of Doom
- ◆ Nested Callback Problem

Why Does Callback Hell Occur?

JavaScript is single-threaded. (One task at a time)

To handle time-consuming tasks, it uses **asynchronous programming**.

When tasks depend on each other, developers often write:

- Callback inside callback
- Inside another callback
- And so on

This creates a pyramid structure.

Example of Callback Hell

JavaScript ▾

```
1 setTimeout(() => {
2   console.log("Step 1: Login");
3
4   setTimeout(() => {
5     console.log("Step 2: Fetch User Data");
6
7     setTimeout(() => {
8       console.log("Step 3: Fetch Orders");
9
10      setTimeout(() => {
11        console.log("Step 4: Payment Done");
12      }, 1000);
13    }, 1000);
14  }, 1000);
15
16 }, 1000);
17
18 }, 1000);
19
```

Output

Step 1: Login 

Step 2: Fetch User Data 

Step 3: Fetch Orders 

Step 4: Payment Done 

Problems of Callback Hell

1. Poor Readability

Code becomes confusing due to deep nesting.

2. Difficult Debugging

Finding where an error occurs is harder.

3. Complex Error Handling

Each callback needs separate error handling.

4. Hard to Maintain

Updating or modifying nested code becomes difficult.

How to Avoid Callback Hell

Use Promises

```
JavaScript ▾  
1 const myPromise = new Promise((resolved, rejected) => {  
2   let marks = 70;  
3   setTimeout(() => {  
4     if (marks >= 50) {  
5       resolved("Student Passed ✅");  
6     } else {  
7       rejected("Student Failed ❌");  
8     }  
9   }, 1000);  
10 })  
11 .then((result) => {  
12   console.log(result);  
13 })  
14 .catch((error) => {  
15   console.log(error);  
16 });  
17
```

Use Async/Await

```
JavaScript  ▾  
1  const myPromise = new Promise((resolved, rejected) => {  
2      let marks = 50;  
3      setTimeout(() => {  
4          if (marks >= 50) {  
5              resolved("Student Passed ✅");  
6          } else {  
7              rejected("Student Failed ❌");  
8          }  
9      }, 1000);  
10 });  
11 async function checkResult() {  
12     try {  
13         const result = await myPromise;  
14         console.log(result);  
15     } catch (error) {  
16         console.log(error);  
17     }  
18     console.log("hello")  
19 }  
20 checkResult();  
21
```

Output

```
Student Passed ✅  
hello
```

4. How do JavaScript timing events (setTimeout and setInterval) work, what are their differences, and Explain their implementation with examples?

Timer Functions in JavaScript

Timer functions are used to execute code after a specific delay or repeatedly at fixed time intervals. JavaScript provides two main timer functions:

1. **setTimeout()**

2. setInterval()

1) setTimeout() Function

► Definition:

setTimeout() is used to execute a function **once** after a specified time delay (in milliseconds).

► Syntax:

javascript

 Copy code

```
setTimeout(function, delay, parameter1, parameter2, ...);
```

- `function` → The function to execute
- `delay` → Time in milliseconds (1000 ms = 1 second)

► Example Program:

javascript

 Copy code

```
function greet() {  
    console.log("Hello after 3 seconds!");  
}  
  
setTimeout(greet, 3000);
```

► Output:

After 3 seconds, it prints:

nginx

 Copy code

```
Hello after 3 seconds!
```

2) setInterval() Function

► Definition:

setInterval() is used to execute a function **repeatedly** at specified time intervals.

► Syntax:

javascript

 Copy code

```
setInterval(function, interval);
```

- `function` → The function to execute
 - `interval` → Time in milliseconds
-

► Example Program:

javascript

 Copy code

```
function showTime() {  
    console.log("Running every 2 seconds");  
}  
  
setInterval(showTime, 2000);
```

► Output:

It prints every 2 seconds continuously:

sql

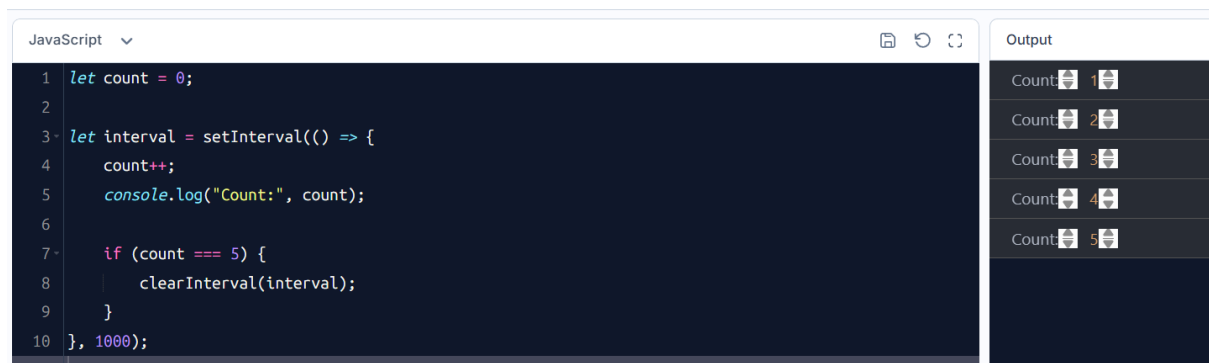
 Copy code

```
Running every 2 seconds  
Running every 2 seconds  
Running every 2 seconds  
...
```



Note: Since the `setInterval()` method executes the function infinitely hence there is a method called as `clearInterval()` to stop the execution of the `setInterval()`.

Example using `clearInterval()` to stop the execution of the `setInterval()`.



```
JavaScript
1 let count = 0;
2
3 let interval = setInterval(() => {
4   count++;
5   console.log("Count:", count);
6
7   if (count === 5) {
8     clearInterval(interval);
9   }
10 }, 1000);
```

Output

Count 1

Count 2

Count 3

Count 4

Count 5

5. Explain the concept of “Pyramid of Doom” in detail along with example program?

Answer:

Callback Hell is a situation where multiple asynchronous operations are nested inside one another using callbacks, resulting in deeply indented and hard-to-read code.

It is also called:

- ◆ Pyramid of Doom
- ◆ Nested Callback Problem

Why Does Callback Hell Occur?

JavaScript is single-threaded. (One task at a time)

To handle time-consuming tasks, it uses **asynchronous programming**.

When tasks depend on each other, developers often write:

- Callback inside callback
- Inside another callback
- And so on

This creates a pyramid structure.

Example of Callback Hell

JavaScript ▾

```
1 setTimeout(() => {  
2   console.log("Step 1: Login");  
3  
4   setTimeout(() => {  
5     console.log("Step 2: Fetch User Data");  
6  
7     setTimeout(() => {  
8       console.log("Step 3: Fetch Orders");  
9  
10      setTimeout(() => {  
11        console.log("Step 4: Payment Done");  
12      }, 1000);  
13    }, 1000);  
14  }, 1000);  
15  
16  }, 1000);  
17  
18 }, 1000);  
19
```

Output

Step 1: Login 

Step 2: Fetch User Data 

Step 3: Fetch Orders 

Step 4: Payment Done 

Problems of Callback Hell

1. Poor Readability

Code becomes confusing due to deep nesting.

2. Difficult Debugging

Finding where an error occurs is harder.

3. Complex Error Handling

Each callback needs separate error handling.

4. Hard to Maintain

Updating or modifying nested code becomes difficult.

How to Avoid Callback Hell

Use Promises

```
JavaScript ▾  
1 const myPromise = new Promise((resolved, rejected) => {  
2   let marks = 70;  
3   setTimeout(() => {  
4     if (marks >= 50) {  
5       resolved("Student Passed ✅");  
6     } else {  
7       rejected("Student Failed ❌");  
8     }  
9   }, 1000);  
10 })  
11 .then((result) => {  
12   console.log(result);  
13 })  
14 .catch((error) => {  
15   console.log(error);  
16 });  
17
```

Use Async/Await

```
JavaScript  ▾  
1  const myPromise = new Promise((resolved, rejected) => {  
2    let marks = 50;  
3    setTimeout(() => {  
4      if (marks >= 50) {  
5        resolved("Student Passed ✅");  
6      } else {  
7        rejected("Student Failed ❌");  
8      }  
9    }, 1000);  
10  });  
11  async function checkResult() {  
12    try {  
13      const result = await myPromise;  
14      console.log(result);  
15    } catch (error) {  
16      console.log(error);  
17    }  
18    console.log("hello")  
19  }  
20  checkResult();  
21
```

Output

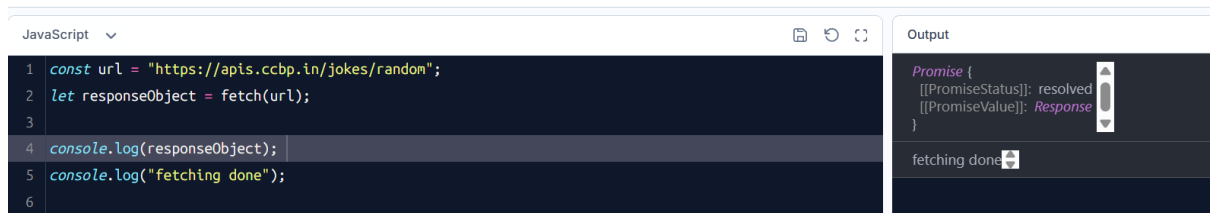
```
Student Passed ✅  
hello
```

6. How do Promises manage asynchronous operations, and what is their constructor structure and Explain briefly with an example?

Answer:

JS Promises

1. Promise is a way to handle **Asynchronous** operations.
2. A promise is an object that represents a result of operation that will be returned at some point in the future.



The screenshot shows a code editor with the following JavaScript code:

```
1 const url = "https://apis.ccbp.in/jokes/random";
2 let responseObject = fetch(url);
3
4 console.log(responseObject);
5 console.log("fetching done");
6
```

The output on the right shows:

```
Promise {
  [[PromiseStatus]]: resolved
  [[PromiseValue]]: Response
}
fetching done
```

States of a Promise

A Promise has three states:

1. **Pending** – Initial state (operation is not completed)
2. **Fulfilled (Resolved)** – Operation completed successfully
3. **Rejected** – Operation failed

3 Structure of a Promise

The basic structure of a Promise:



The screenshot shows a code editor with the following JavaScript code:

```
javascript Copy code

let promise = new Promise(function(resolve, reject) {
  // asynchronous task

  if (success) {
    resolve("Success message");
  } else {
    reject("Error message");
  }
});
```

Explanation:

- `new Promise()` → Creates a Promise object
- `resolve()` → Called when operation is successful
- `reject()` → Called when operation fails
- `.then()` → Handles success
- `.catch()` → Handles error

Example: -

JavaScript ▾

```
i 1- const myPromise = new Promise((resolved, rejected) => {
2    let marks = 70;
3-   setTimeout(() => {
4-       if (marks >= 50) {
5           resolved("Student Passed ✅");
6-       } else {
7           rejected("Student Failed ❌");
8       }
9   }, 1000);
10 })
11- .then((result) => {
12     console.log(result);
13 })
14- .catch((error) => {
15     console.log(error);
16 });
17
```

MODULE III

Short Answers

1. How does JSX bridge the gap between HTML markup and JavaScript logic in React?

JSX (JavaScript XML) acts as a bridge between **HTML structure** and **JavaScript logic** in React by allowing developers to write UI components using a syntax that looks like HTML but works inside JavaScript.

How JSX bridges the gap

1. HTML-like Syntax inside JavaScript

- JSX lets you write UI elements in a familiar HTML-like format.
- This makes the code more readable and easier to design layouts.

```
</> JavaScript
const element = <h1>Hello World</h1>
```



2. Embedding JavaScript Expressions

- You can directly use JavaScript inside JSX using `{}`.
- This connects dynamic logic with UI.

```
</> JavaScript
const name = "Shaik"
const element = <h1>Hello, {name}</h1>
```



3. Conditional Rendering

- JSX allows conditions using JavaScript operators.

```
</> JavaScript
const isLoggedIn = true
const element = <h1>{isLoggedIn ? "Welcome" : "Please Login"}</h1>
```



4. Event Handling Integration

- JSX allows attaching JavaScript functions to UI elements.

JavaScript

```
<button onClick={handleClick}>Click Me</button>
```



5. Converts to JavaScript (Behind the Scenes)

- JSX is not understood by browsers directly.
- It gets compiled into `React.createElement()` calls.

JavaScript

```
// JSX
```

```
const element = <h1>Hello</h1>
```

```
// Converted JS
```

```
const element = React.createElement("h1", null, "Hello")
```



2. Explain the core concepts of React and its significance in modern front-end development?

◆ Core Concepts of React

1. Components

- React applications are built using **components** (reusable, independent pieces of UI).
- Two types: **Functional components** (modern) and **Class components** (older).

2. JSX (JavaScript XML)

- JSX allows writing **HTML-like code inside JavaScript**.
- Makes UI code more readable and easier to maintain.

3. Props (Properties)

- Props are used to **pass data from parent to child components**.
- They are **read-only (immutable)**.

4. State

- State is used to **store and manage dynamic data** within a component.
- When state changes, the component **re-renders automatically**.

5. Virtual DOM

- React uses a **Virtual DOM** (a lightweight copy of the real DOM).
- It updates only the changed parts using a **diffing algorithm**, improving performance.

6. Event Handling

- React handles events (like clicks, inputs) using **JavaScript functions**.
- Similar to DOM events but follows React's syntax.

7. Hooks

- Hooks (e.g., `useState`, `useEffect`) allow using **state and lifecycle features in functional components**.
- Simplifies code and avoids class components.

8. One-Way Data Binding

- Data flows **from parent to child**.
- Makes applications more predictable and easier to debug.

◆ Significance in Modern Front-End Development

1. Reusability

- Components can be reused, reducing code duplication and development time.

2. High Performance

- Virtual DOM ensures **efficient updates** and faster rendering.

3. Better Maintainability

- Modular structure makes code easier to manage and scale.

4. Strong Ecosystem

- Supported by tools like **React Router**, etc.

5. Declarative Approach

- Developers describe *what UI should look like*, and React handles updates.

6. Industry Demand

- Widely used in modern applications (Facebook, Instagram, Netflix), making it a **high-demand skill**.

3. What role does JSX play in React development, and how does it simplify component creation?

Role of JSX in React Development

JSX (JavaScript XML) is a syntax extension that allows developers to write **HTML-like code inside JavaScript**. It plays a central role in React by making UI creation **more intuitive and declarative**.

◆ How JSX Simplifies Component Creation

1. Readable and Familiar Syntax

- JSX looks like HTML, so it is easy to understand and write.
- Developers can design UI structure clearly within the component.

```
</> JavaScript   
  
function Welcome() {  
  return <h1>Hello, User</h1>  
}
```

2. Combines UI and Logic in One Place

- JSX allows embedding JavaScript expressions using {}.
- This removes the need to separate logic and markup.

```
</> JavaScript   
  
function Welcome(props) {  
  return <h1>Hello, {props.name}</h1>  
}
```

3. Supports Dynamic Content

- You can easily render dynamic data, conditions, and lists.

```
</> JavaScript   
  
const isLoggedIn = true  
return <h1>{isLoggedIn ? "Welcome" : "Please Login"}</h1>
```

4. Simplifies Component Structure

- Instead of using complex DOM methods (createElement), JSX provides a cleaner way to define UI.

```
</> JavaScript   
  
// Without JSX  
React.createElement("h1", null, "Hello")  
  
// With JSX  
<h1>Hello</h1>
```

5. Improves Developer Productivity

- Less boilerplate code

- Easier debugging and maintenance
- Faster development of components

6. Enables Declarative UI

- Developers describe *what the UI should look like*, and React handles updates automatically.

4. Explain the role of the Virtual DOM in updating user interfaces efficiently?

The **Virtual DOM** is a lightweight copy of the real DOM (web page structure) stored in memory. It helps React update the user interface **quickly and efficiently**.

◆ Role in Updating UI Efficiently

1. Creates a Virtual Copy

- React first creates a **Virtual DOM** instead of directly working with the real DOM.

2. Detects Changes

- When data (state/props) changes, React creates a **new Virtual DOM**.

3. Compares Changes (Diffing)

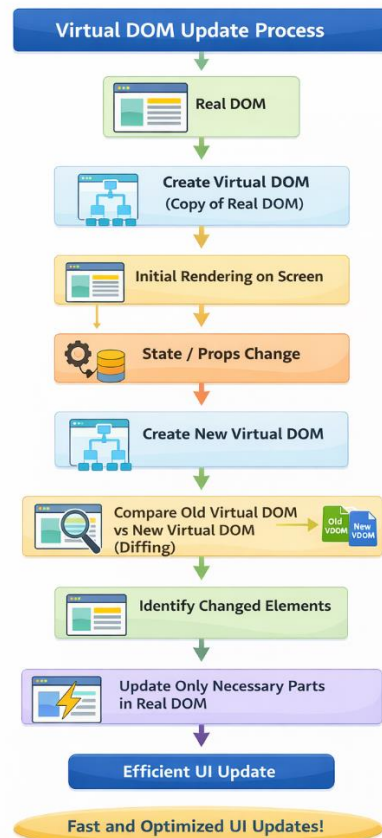
- React compares the **old Virtual DOM** with the **new one**.
- It finds only what has changed.

4. Updates Only Required Parts

- Instead of updating the entire page, React updates **only the changed elements** in the real DOM.

◆ Why This is Efficient

- Real DOM updates are **slow**
- Virtual DOM reduces unnecessary updates
- Makes applications **faster and smoother**



5. What are the differences between functional components and class components in React?

Feature	Functional Components	Class Components
Definition	Simple JavaScript functions	ES6 classes that extend React.Component
Syntax	Short and simple	More complex and verbose
State Handling	Uses Hooks (useState, useEffect)	Uses this.state and this.setState()
Lifecycle Methods	Managed using Hooks (useEffect)	Uses lifecycle methods (componentDidMount, etc.)
this Keyword	Not used	Must use this keyword
Performance	Generally faster and lighter	Slightly heavier
Code Readability	Easier to read and maintain	More complex to understand

Boilerplate Code	Less code	More boilerplate required
Modern Usage	Preferred in modern React	Less commonly used now
Example	<pre>function App() { return <h1>Hello</h1> }</pre>	<pre>class App extends React.Component { render() { return <h1>Hello</h1> } }</pre>

6. Explain the role of props in React and why a child component cannot change its own props?

Role of Props in React

Props (Properties) are used to **pass data from a parent component to a child component**.

Key Roles:

- **Data Transfer:** Props allow communication between components.
- **Reusability:** Same component can display different data using different props.
- **Read-Only Data:** Props provide fixed input to components.

```

</> JavaScript
function Child(props) {
  return <h1>Hello, {props.name}</h1>
}

```

◆ Why Child Component Cannot Change Props

1. Props are Immutable (Read-Only)

- Props cannot be modified by the child component.
- They are controlled only by the **parent component**.

2. One-Way Data Flow

- React follows **unidirectional data flow** (Parent → Child).
- This keeps the data predictable and easier to debug.

3. Maintain Data Consistency

- If children could change props, it would create confusion and bugs.
- Keeping props read-only ensures **stable and consistent UI**.

4. Separation of Responsibilities

- Parent manages data.

- Child only **uses and displays data**.

Long Answers:

1. What are the core principles and features of the React library Explain with an Example?

Core Principles and Features of React

React is a **JavaScript library** used to build **fast and interactive user interfaces**.

◆ Core Principles / Features

1. Component-Based Architecture

- React divides UI into **small reusable components**.
- Each component works independently.

2. JSX (JavaScript XML)

- Allows writing **HTML-like code inside JavaScript**.
- Makes code easy to read and write.

3. Declarative UI

- You describe how the UI should look and React updates it automatically.

4. Virtual DOM

- React uses a **Virtual DOM** to update only changed parts.
- This makes applications faster.

5. One-Way Data Flow

- Data flows from **parent to child** using props.
- Makes debugging easier.

6. Hooks (Modern Feature)

- Hooks like **useState** help manage data in functional components.

```

src > App.js > ...
1  import { useState } from "react";
2
3  const Counter = () => {
4    const [count, setCount] = useState(0);
5
6    const onIncrement = () => {
7      setCount((prevCount) => prevCount + 1);
8    };
9
10   const onDecrement = () => {
11     setCount((prevCount) => prevCount - 1);
12   };
13
14   return (
15     <div>
16       <h1>Counter</h1>
17       <p>{count}</p>
18       <div>
19         <button onClick={onIncrement}>Increase</button>
20         <button onClick={onDecrement}>Decrease</button>
21       </div>
22     </div>
23   );
24 };
25
26 export default Counter;
27

```

2. What are the different ways to create components in React and when should I use them Explain with a suitable Example?

There are two main types of components in React:

A Functional Component in React is a simple JavaScript function that returns JSX (UI).

A Class Component in React is a component created using a JavaScript class.

Functional Components

- Simple **JavaScript functions**
- Return **JSX** to display UI
- Use **Hooks** (useState, useEffect) for state and logic
- Do **not use this keyword**
- Easy to write and understand
- Less code (less boilerplate)
- Preferred in **modern React development**

- Better for **reusability and maintainability**

1. Functional Component (with Props & State)

- Uses **functions + Hooks (useState)**
- Accepts **props** and manages **state**

Example:

```
JavaScript   
  
import { useState } from "react";  
  
function Counter(props) {  
  const [count, setCount] = useState(0);  
  
  return (  
    <div>  
      <h2>{props.title}</h2>  
      <p>Count: {count}</p>  
      <button onClick={() => setCount(count + 1)}>  
        Increment  
      </button>  
    </div>  
  );  
}
```

Class Components

- Created using **ES6 classes**
- Extend `React.Component`
- Use **render() method** to return JSX
- Manage state using **this.state**
- Update state using **this.setState()**
- Use **lifecycle methods** (`componentDidMount`, etc.)
- Use **this keyword**
- More complex and verbose
- Mainly used in **legacy (old) React projects**

2. Class Component (with Props & State)

- Uses **ES6 class**, `this.state` and `this.setState()`
- Access props using `this.props`

```
App.js  X
src > App.js > Counter
1  import React from "react";
2
3  class Counter extends React.Component {
4    constructor(props) {
5      super(props);
6      this.state = { count: 0 };
7    }
8
9    increment = () => {
10     this.setState({ count: this.state.count + 1 });
11   };
12
13   render() {
14     return (
15       <div>
16         <h2>{this.props.title}</h2>
17         <p>Count: {this.state.count}</p>
18         <button onClick={this.increment}>Increment</button>
19       </div>
20     );
21   }
22 }
```

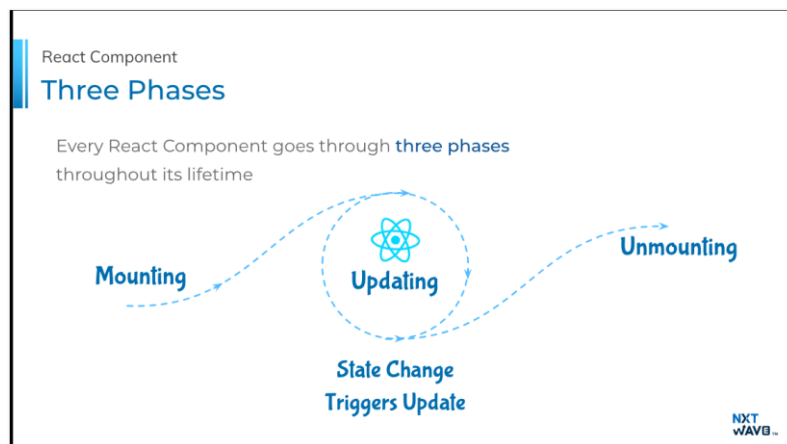
3. Compare functional components and class components in React with examples.

Feature	Functional Components	Class Components
Definition	Simple JavaScript functions	ES6 classes that extend React.Component
Syntax	Short and simple	More complex and verbose
State Handling	Uses Hooks (useState, useEffect)	Uses this.state and this.setState()
Lifecycle Methods	Managed using Hooks (useEffect)	Uses lifecycle methods (componentDidMount, etc.)
this Keyword	Not used	Must use this keyword
Performance	Generally faster and lighter	Slightly heavier
Code Readability	Easier to read and maintain	More complex to understand

Boilerplate Code	Less code	More boilerplate required
Modern Usage	Preferred in modern React	Less commonly used now
Example	<pre>function App() { return <h1>Hello</h1> }</pre>	<pre>class App extends React.Component { render() { return <h1>Hello</h1> } }</pre>

4. Explain the Component Lifecycle in detail, covering the Mounting, Updating, and Unmounting phases.

Every React component goes through **three main phases** during its lifetime:

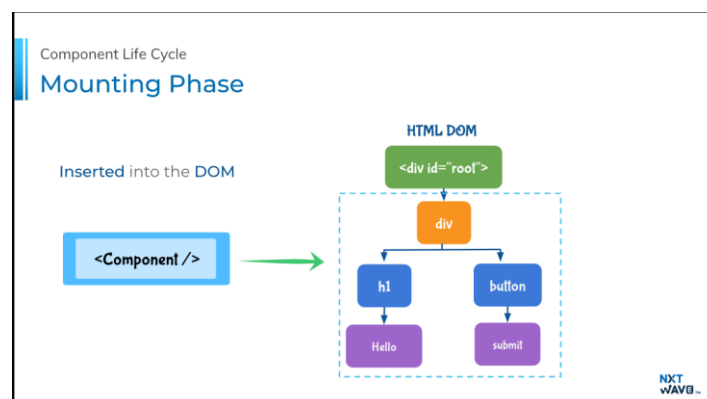


◆ 1. Mounting Phase

- The component is **created and added to the DOM**
- Initial UI is displayed

✓ Example (Clock):

When the Clock component is shown, it starts displaying the current time.



◆ 2. Updating Phase

- Occurs when **state or props change**
- Component **re-renders (updates UI)**

✓ Example:

Using `setInterval()`, the time updates **every second**, so the component re-renders continuously.

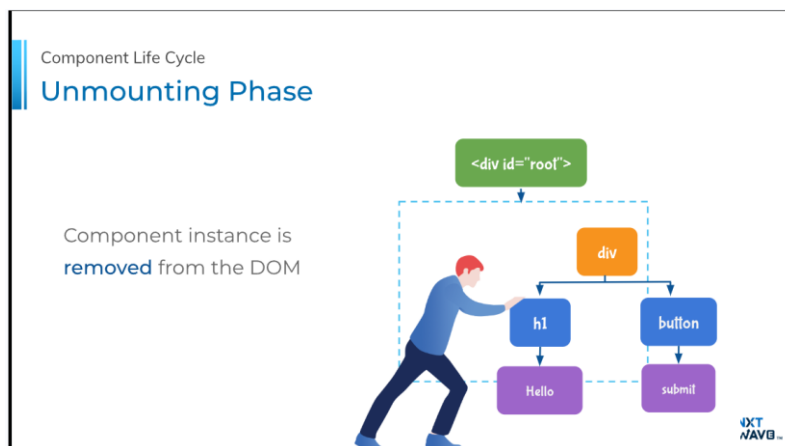


◆ 3. Unmounting Phase

- The component is **removed from the DOM**
- Cleanup actions are performed

✓ Example:

When the Clock is hidden, we stop the timer using `clearInterval()`.



◆ Simple Example (Clock using Hooks)

```
JavaScript

import { useState, useEffect } from "react";

const Clock = () => {
  const [time, setTime] = useState(new Date());

  useEffect(() => {
    const timerId = setInterval(() => {
      setTime(new Date()); // Updating phase
    }, 1000);

    return () => {
      clearInterval(timerId); // Unmounting (cleanup)
    };
  }, []);

  return <h1>{time.toLocaleTimeString()}</h1>;
};

export default Clock;
```

5.What is JSX? Write a code snippet demonstrating how to embed JavaScript expressions and attributes within JSX?

JSX (JavaScript XML) is a syntax extension used in React that allows developers to write **HTML-like code inside JavaScript**. It helps in creating UI components in a **simple, readable, and structured way**.

◆ How JSX Works

- Browsers cannot understand JSX directly
- JSX is converted into **JavaScript (React.createElement())** using a tool called **Babel**
- JSX ultimately creates **React elements**, which React uses to update the UI efficiently using the Virtual DOM

◆ Key Features of JSX

1. HTML-like Syntax

- JSX looks similar to HTML, making it easy to design UI

```
JavaScript

<h1>Hello World</h1>
```

2. Embedding JavaScript Expressions

- Use **curly braces** `{}` to include JavaScript inside JSX

```
</> JavaScript
const name = "Shaik";
<h1>Hello, {name}</h1>
```



3. Attributes in JSX (camelCase)

- HTML attributes are written in **camelCase**
- Example: `class` → `className`, `onclick` → `onClick`

```
</> JavaScript
<div className="box">
  <button onClick={() => alert("Clicked")}>Click</button>
</div>
```



4. Single Parent Element

- JSX must return only **one root element**

```
</> JavaScript
return (
  <div>
    <h1>Title</h1>
    <p>Content</p>
  </div>
);
```



5. Self-Closing Tags

- All tags must be properly closed

```
</> JavaScript

```



◆ Complete Example (Expressions + Attributes)

◆ Car.jsx

```
JavaScript

function Car() {
  const name = "BMW";
  const price = 5000000;

  return (
    <div>
      <h1>Car Name: {name}</h1>
      <p>Original Price: {price}</p>
      <p>Discount Price: {price - 500000}</p>

      <button onClick={() => alert(name + " Selected")}>
        Select Car
      </button>
    </div>
  );
}

export default Car;
```

◆ App.jsx

```
JavaScript

import Car from "./Car";

function App() {
  return (
    <div>
      <Car />
    </div>
  );
}

export default App;
```

Car Name: BMW

Original Price: 5000000

Discount Price: 4500000

Select Car

6. Describe the process of Props Communication between a parent and a child component with a code snippet?

Props Communication in React (ELS10 – Detailed Explanation)

Props (Properties) are used to **pass data from a parent component to a child component**. They are the main way components communicate in React and help make components **reusable and dynamic**.

◆ Step-by-Step Process

1. Parent Creates Data

- The parent component defines some data (variables, values, or functions)

2. Parent Passes Data as Props

- Data is passed to the child using **attributes (like HTML attributes)**

3. Child Receives Props

- The child component receives the data as an object called props

4. Child Uses Props

- The child uses props to display or work with the data

◆ Flow of Data

Parent Component → Child Component
(props)



👉 This is called **Unidirectional Data Flow (One-Way Data Flow)**

◆ Detailed Example

Parent Component (App.jsx)

```
<> JavaScript
import Child from "../Child";

function App() {
  const name = "Shaik";
  const age = 21;

  return (
    <div>
      <Child username={name} userAge={age} />
    </div>
  );
}

export default App;
```



Child Component (Child.jsx)

```
</> JavaScript   
  
function Child(props) {  
  return (  
    <div>  
      <h1>Hello, {props.username}</h1>  
      <p>Age: {props.userAge}</p>  
    </div>  
  );  
}  
  
export default Child;
```

Hello, Shaik

Age: 21

◆ Important Points

- Props are **read-only (immutable)** → Child cannot change them
- Data always flows **Parent → Child**
- Props can pass:
 - ☒ Data (strings, numbers)
 - ☒ Functions
 - ☒ Arrays, objects
- Makes components **reusable** (same component, different data)

MODULE IV

Short Answers

1. How do you add state management capabilities to a functional React component?

◆ Adding State Management in Functional Components

In React, **state management in functional components** is done using **Hooks**, mainly the **useState hook**.

◆ Steps to Add State

1. Import useState Hook

- It is provided by React to manage state in functional components

```
import { useState } from "react";
```

2. Initialize State

- `useState()` returns two values:
 - Current state value
 - Function to update state

```
const [count, setCount] = useState(0);
```

3. Use State in JSX

- Display or use the state value in UI

```
<h1>{count}</h1>
```

4. Update State

- Use the setter function to change state
- When state updates, component **re-renders automatically**

```
<button onClick={() => setCount(count + 1)}> Increment </button>
```

```

JavaScript
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h1>Count: {count}</h1>
      <button onClick={() => setCount(count + 1)}>
        Click Me
      </button>
    </div>
  );
}

export default Counter;

```

◆ Key Points

- useState is used to **manage state in functional components**
- State updates cause **re-rendering of UI**
- Makes components **dynamic and interactive**

2. What constitutes a side effect, and how can you handle it in functional components in React?

◆ What is a Side Effect in React?

A **side effect** is any operation in a component that affects something **outside the component's rendering**.

📖 In simple terms:

Anything that is **not just returning UI (JSX)** is a side effect.

◆ Examples of Side Effects

- Fetching data from an API
- Using setInterval() or setTimeout()
- Updating the DOM manually
- Adding event listeners
- Accessing local storage

◆ How to Handle Side Effects in Functional Components

React provides the **useEffect Hook** to handle side effects.

◆ Steps to Use useEffect

1. Import useEffect

```
import { useEffect } from "react";
```

2. Use useEffect Hook

- It runs **after the component renders**

```
useEffect(() => {  
  // side effect code here  
});
```

3. Control Execution with Dependency Array

- [] → Runs only once (on mounting)
- [value] → Runs when value changes
- No array → Runs on every render

◆ Complete Example

```
«> JavaScript   
  
import { useState, useEffect } from "react";  
  
function Clock() {  
  const [time, setTime] = useState(new Date());  
  
  useEffect(() => {  
    const timerId = setInterval(() => {  
      setTime(new Date()); // side effect  
    }, 1000);  
  
    return () => clearInterval(timerId); // cleanup  
  }, []);  
  
  return <h1>{time.toLocaleTimeString()}</h1>;  
}
```

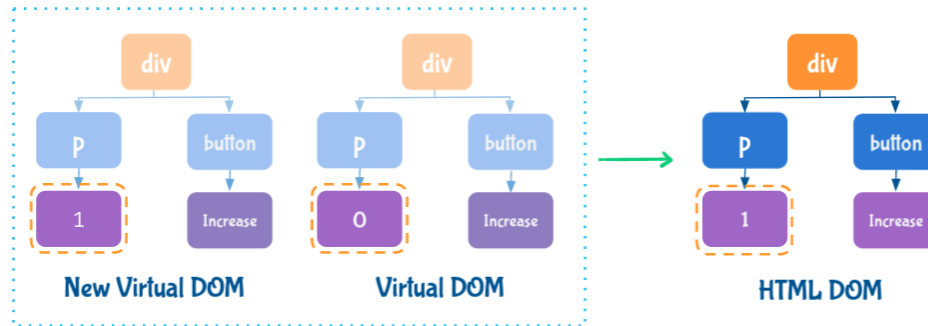
3. What is the role of keys in React's reconciliation (diffing) algorithm?

1. Reconciliation

1.1 Virtual DOM

A Virtual DOM is a JS object, which is the virtual representation of an HTML DOM.

Whenever new elements are added to the UI, a virtual DOM is created.



**NXT
WAVE**

React compares new virtual DOM with current virtual DOM, and the difference will be efficiently updated to HTML DOM. So, the Virtual DOM helps to render the UI more performantly.

React only updates what's necessary. This process is known as **Reconciliation**.

4. Explain the performance and functional tradeoffs between controlled and uncontrolled inputs in React?

Controlled vs Uncontrolled Input

In React, the Input Element value can be handled in two ways:

- Controlled Input
- Uncontrolled Input

5.1 Controlled Input

If the Input Element value is handled by a React State, then it is called **Controlled Input**.

It is the React Suggested way to handle the Input Element value.

```

1 import { useState } from "react";
2
3 const App = () => {
4   const [userInput, setUserInput] = useState("");
5
6   const onChangeUserInput = (event) => {
7     setUserInput(event.target.value);
8   };
9
10  return (
11    <>
12      <input type="text" onChange={onChangeUserInput} value={userInput} />
13      <p>{userInput}</p>
14    </>
15  );
16 };
17
18 export default App;

```

5.2 Uncontrolled Input

If the Input Element value is handled by the browser itself, then it is called **Uncontrolled Input**.

Example:

```
1 import { useState } from "react";
2
3 const App = () => {
4   const [userInput, setUserInput] = useState("");
5
6   const onChangeUserInput = (event) => {
7     setUserInput(event.target.value);
8   };
9
10  return (
11    <>
12      <input type="text" onChange={onChangeUserInput} />
13      <p>{userInput}</p>
14    </>
15  );
16 };
17
18 export default App;
```

5. What are the cleanup steps for useEffect Hook to prevent memory leaks?

Cleanup Steps for useEffect Hook

In React, **cleanup steps** are used to **stop or remove side effects** when a component is unmounted or before the effect runs again. This helps to **prevent memory leaks**.

◆ Why Cleanup is Needed

- To stop **timers** (setInterval, setTimeout)
- To remove **event listeners**
- To cancel **API requests**
- To avoid unnecessary background processes

◆ How to Do Cleanup

- Inside useEffect, return a **cleanup function**

```

</> JavaScript
useEffect(() => {
  // side effect

  return () => {
    // cleanup code
  };
}, []);

```

◆ Example (Clearing Interval)

```

</> JavaScript

import { useEffect, useState } from "react";

function Clock() {
  const [time, setTime] = useState(new Date());

  useEffect(() => {
    const timerId = setInterval(() => {
      setTime(new Date());
    }, 1000);

    return () => {
      clearInterval(timerId); // cleanup
    };
  }, []);

  return <h1>{time.toLocaleTimeString()}</h1>;
}

```

Long Answers

1. Explain the **useReducer** hook and how it differs from **useState** for complex state logic?

◆ What is **useReducer** Hook?

useReducer is a React Hook used to **manage complex state logic**.

It works similar to a **reducer function** where state changes are handled using **actions**.

👉 It is useful when:

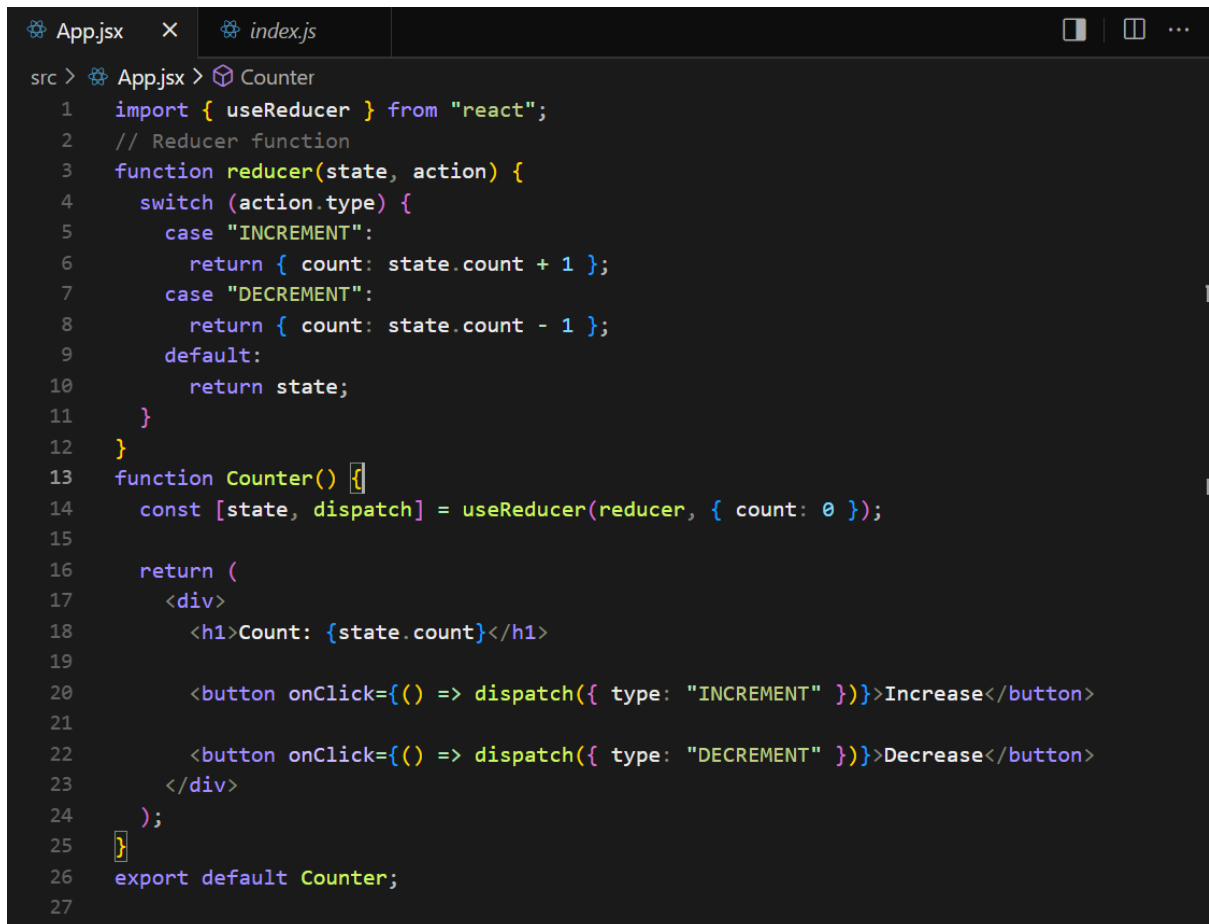
- State logic is complex
- Multiple state updates are needed
- Next state depends on previous state

◆ Syntax

JavaScript

```
const [state, dispatch] = useReducer(reducer, initialState);
```

- `state` → current value
- `dispatch` → function to send actions
- `reducer` → function that updates state



```
App.jsx x index.js
src > App.jsx > Counter
1  import { useReducer } from "react";
2  // Reducer function
3  function reducer(state, action) {
4    switch (action.type) {
5      case "INCREMENT":
6        return { count: state.count + 1 };
7      case "DECREMENT":
8        return { count: state.count - 1 };
9      default:
10       return state;
11    }
12  }
13  function Counter() {
14    const [state, dispatch] = useReducer(reducer, { count: 0 });
15
16    return (
17      <div>
18        <h1>Count: {state.count}</h1>
19
20        <button onClick={() => dispatch({ type: "INCREMENT" })}>Increase</button>
21
22        <button onClick={() => dispatch({ type: "DECREMENT" })}>Decrease</button>
23      </div>
24    );
25  }
26  export default Counter;
27
```

◆ Explanation

- `reducer()` → decides how state changes
- `dispatch()` → sends action (INCREMENT, DECREMENT)
- State is stored as an object → `{ count: 0 }`
- UI updates automatically when state changes

◆ Difference: useState vs useReducer

useState	useReducer
Simple state logic	Complex state logic
Direct update (setCount)	Uses dispatch and reducer
Easy to use	More structured
Best for small apps	Best for large/complex apps

2. Demonstrate how to create and implement a Custom Hook to reuse logic across components?

◆ What is a Custom Hook?

A **Custom Hook** is a reusable JavaScript function that starts with **use** and allows you to reuse logic across multiple components.

◆ Steps to Create a Custom Hook

1. Create a function starting with use
2. Use built-in hooks inside it (useState, etc.)
3. Return required data/functions
4. Use it inside components

◆ Example: Custom Hook for Counter Logic

Step 1: Create Custom Hook (useCounter.js)

Step 1: Create Custom Hook (useCounter.js)

```
JavaScript
import { useState } from "react";

function useCounter(initialValue = 0) {
  const [count, setCount] = useState(initialValue);

  const increment = () => setCount(prev => prev + 1);
  const decrement = () => setCount(prev => prev - 1);

  return { count, increment, decrement };
}

export default useCounter;
```

Step 2: Use Custom Hook in Component 1

```
JavaScript

import useCounter from "../useCounter";

function CounterOne() {
  const { count, increment, decrement } = useCounter();

  return (
    <div>
      <h2>Counter One: {count}</h2>
      <button onClick={increment}>+</button>
      <button onClick={decrement}>-</button>
    </div>
  );
}

export default CounterOne;
```

Step 3: Use Custom Hook in Component 2

```
JavaScript

import useCounter from "../useCounter";

function CounterTwo() {
  const { count, increment } = useCounter(10);

  return (
    <div>
      <h2>Counter Two: {count}</h2>
      <button onClick={increment}>Increase</button>
    </div>
  );
}

export default CounterTwo;
```

◆ Explanation

- useCounter contains reusable logic (state + functions)
- Multiple components can use the same logic
- Each component gets its **own independent state**

◆ Benefits

- ☒ Code reusability
- ☒ Cleaner components
- ☒ Better separation of logic

-  Easy to maintain

◆ Conclusion

Custom Hooks allow you to **extract and reuse logic** across components, making React code more **organized, reusable, and efficient**.

3. Describe the **useEffect** Hook in detail, specifically how the dependency array controls the execution of side effects?

1. Effect Hook

React provides a built-in hook called **useEffect** that allows executing logic after the component render

In general this **useEffect** hook can be simply called as **Effect hook**

Using Effect Hook we can perform actions like,

- Making API Call
- Using Scheduler methods like `setInterval()`
- DOM Manipulations, etc.

◆ `useEffect` Syntax

```
</> JavaScript
import { useEffect } from "react";

useEffect(() => {
  // side effect code

  return () => {
    // cleanup code (optional)
  };
}, [dependencies]);
```

The `useEffect` accepts **effect** as an argument, where the effect is a **callback function**

The `useEffect` accepts another argument called **Dependency Array**, using which we can control the execution of effect

Syntax:

```
1 useEffect(effect, [var1, var2, ...])
```

JSX

- The values we pass to the dependency array are called dependencies, which can be state variables or props
- Effect will be executed whenever the dependencies are changed

Without Dependencies

The Effect executes after every render

```
1 useEffect(effect);
```

JSX

```
</> JavaScript
useEffect(() => {
  console.log("Runs every time");
});
```



Passing Empty Dependency Array

The Effect executes only once after the first render

```
1 useEffect(effect, []);
```

JSX

```
</> JavaScript
useEffect(() => {
  console.log("Runs once");
}, []);
```



Passing Dependencies

The Effect executes when dependencies are changed

```
1 useEffect(effect, [var1, var2,...])
```

JSX

```
</> JavaScript
useEffect(() => {
  console.log("Runs when count changes");
}, [count]);
```

4. Discuss Conditional Rendering techniques in React (e.g., Ternary operators, Logical &&)?

Conditional Rendering allows us to render different elements or components based on a condition.

Different ways to implement **Conditional Rendering** are:

- Using an If...Else Statement
- Using Element Variables
- Using Ternary Operators
- Using Logical && Operator

1.1 Using an If...Else Statement



```
1 import { useState } from "react";
2
3 import Welcome from "../components/Welcome";
4 import "../App.css";
5
6 const App = () => {
7   const [isLoggedIn, setIsLoggedIn] = useState(true);
8
9   const renderAuthButton = () => {
10     if (isLoggedIn === true) {
11       return <button>Logout</button>;
12     }
13     return <button>Login</button>;
14   };
15
16   return (
17     <div className="container">
18       <Welcome greeting="Hello" name="User" />
19       {renderAuthButton()}
20     </div>
21   );
22 };
23
24 export default App;
```

The screenshot shows a code editor with a dark theme. The code is a React component named App. It uses the useState hook to manage a boolean state 'isLoggedIn' which is initially true. A function 'renderAuthButton' is defined to conditionally render either a 'Logout' button or a 'Login' button based on the 'isLoggedIn' state. The main render function returns a div with a class 'container' containing a 'Welcome' component and the 'renderAuthButton' function. The code is numbered from 1 to 24. There is a 'JSX' tab in the top right corner and a 'Collapse' button in the bottom right corner.

1.2 Using Element Variables

```
1 import { useState } from "react";
2
3 import Welcome from "../components/Welcome";
4 import "./App.css";
5
6 const App = () => {
7   const [isLoggedIn, setIsLoggedIn] = useState(true);
8
9   let authButton;
10  if (isLoggedIn === true) {
11    authButton = <button>Logout</button>;
12  } else {
13    authButton = <button>Login</button>;
14  }
15
16  return (
17    <div className="container">
18      <Welcome greeting="Hello" name="User" />
19      {authButton}
20    </div>
21  );
22 };
23
24 export default App;
```

JSX

Collapse ^

1.3 Using Ternary Operators

```
1 import { useState } from "react";
2
3 import Welcome from "../components/Welcome";
4 import "./App.css";
5
6 const App = () => {
7   const [isLoggedIn, setIsLoggedIn] = useState(true);
8
9   return (
10     <div className="container">
11       <Welcome greeting="Hello" name="User" />
12       {isLoggedIn ? <button>Logout</button> : <button>Login</button>}
13     </div>
14   );
15 };
16
17 export default App;
```

JSX

Collapse ^

1.4 Using Logical && Operator

```
1 import { useState } from "react";
2
3 import Welcome from "../components/Welcome";
4 import "./App.css";
5
6 const App = () => {
7   const [isLoggedIn, setIsLoggedIn] = useState(true);
8
9   return (
10     <div className="container">
11       <Welcome greeting="Hello" name="User" />
12       {isLoggedIn && <button>Logout</button>}
13       {!isLoggedIn && <button>Login</button>}
14     </div>
15   );
16 };
17
18 export default App;
```

JSX

Collapse ^

5. Discuss Event Handling Techniques in React. How do you pass arguments to an event handler without triggering it on render?

◆ Event Handling in React

Event handling in React allows components to **respond to user actions** such as clicks, typing, form submission, mouse events, etc. React events are similar to DOM events but follow **camelCase syntax** and use **functions instead of strings**.

◆ Basic Concept

- Events are written as **attributes in JSX**
- Event names use **camelCase** (onClick, onChange, onSubmit)
- You pass a **function** as the event handler

JavaScript

```
<button onClick={handleClick}>Click Me</button>
```



◆ Different Event Handling Techniques

1. Inline Event Handling

- Function is written directly inside JSX

JavaScript

```
<button onClick={() => alert("Clicked")}>
  Click Me
</button>
```



👉 Simple but not recommended for complex logic

2. Using Separate Function (Best Practice)

- Define function outside and pass reference

JavaScript

```
function handleClick() {
  alert("Button Clicked");
}

<button onClick={handleClick}>Click Me</button>
```



👉 Cleaner and reusable

3. Event Handling with State Update

- Events can modify component state

```
JavaScript

import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Count: {count}
    </button>
  );
}
```

◆ Passing Arguments to Event Handler

✗ Wrong Way (Executes Immediately)

```
JavaScript

<button onClick={handleClick(5)}>Click</button>
```

👉 Problem:

- Function runs **during rendering**, not on click

◆ Correct Ways

1. Using Arrow Function (Most Common)

```
JavaScript

<button onClick={() => handleClick(5)}>
  Click Me
</button>
```

👉 Function runs **only when clicked**

6. What are the best practices for handling form submission and input data in React Explain with Suitable Example?

◆ Handling Form Submission and Input Data in React

Handling forms in React involves managing **input values using state** and handling submission using event handlers.

◆ Best Practices

1. Use Controlled Components

- Store input values in **state**
 - Update state using `onChange`
-

2. Use `onChange` for Inputs

- Keeps UI and state in sync
-

3. Prevent Default Form Submission

- Use `event.preventDefault()` to stop page reload
-

4. Handle Multiple Inputs Efficiently

- Use `name` attribute and a single handler
-

5. Validate Input Data

- Check input values before submitting

Controlled vs Uncontrolled Input

In React, the Input Element value can be handled in two ways:

- Controlled Input
- Uncontrolled Input

5.1 Controlled Input

If the Input Element value is handled by a React State, then it is called **Controlled Input**.

It is the React Suggested way to handle the Input Element value.

```
1 import { useState } from "react";
2
3 const App = () => {
4   const [userInput, setUserInput] = useState("");
5
6   const onChangeUserInput = (event) => {
7     setUserInput(event.target.value);
8   };
9
10  return (
11    <>
12      <input type="text" onChange={onChangeUserInput} value={userInput} />
13      <p>{userInput}</p>
14    </>
15  );
16 };
17
18 export default App;
```

5.2 Uncontrolled Input

If the Input Element value is handled by the browser itself, then it is called **Uncontrolled Input**.

Example:

```
1 import { useState } from "react";
2
3 const App = () => {
4   const [userInput, setUserInput] = useState("");
5
6   const onChangeUserInput = (event) => {
7     setUserInput(event.target.value);
8   };
9
10  return (
11    <>
12      <input type="text" onChange={onChangeUserInput} />
13      <p>{userInput}</p>
14    </>
15  );
16 };
17
18 export default App;
```

MODULE-V

SHORT ANSWERS

1. What is the role of BrowserRouter and Route in React Router?

Role of BrowserRouter and Route in React Router

React Router is used to create **navigation between different pages** in a React application without reloading the page.

◆ 1. BrowserRouter

- BrowserRouter is the **main router component**
- It **wraps the entire application**
- It uses the **browser's history API** to manage navigation
- Enables **client-side routing**

☑ Role:

- Keeps UI in sync with URL
 - Provides routing functionality to all child components
-

◆ 2. Route

- Route is used to **define paths (URLs)**
- It tells React **which component to render** for a specific path

☑ Role:

- Matches the URL
- Renders the corresponding component

```

src > App.jsx > ...
1  import { BrowserRouter, Routes, Route } from "react-router";
2
3  function Home() {
4    return <h1>Home Page</h1>;
5  }
6
7  function About() {
8    return <h1>About Page</h1>;
9  }
10
11 function App() {
12   return (
13     <BrowserRouter>
14       <Routes>
15         <Route path="/" element={<Home />} />
16         <Route path="/about" element={<About />} />
17       </Routes>
18     </BrowserRouter>
19   );
20 }
21
22 export default App;
23

```

2. How do you extract values using the useParams() hook?

2.1 useParams

The useParams hook allows to access the path parameters from the current rendered route URL

Syntax

```
1 const { parameter } = useParams()
```

JSX

It returns an **object** that contains a **Parameter** and its corresponding value in the form of a **key/value** pair

```
App.jsx  X  index.js
src > App.jsx > ...
1  import { BrowserRouter, Routes, Route } from "react-router";
2  import { useParams } from "react-router-dom";
3
4  function User() {
5    const { id } = useParams();
6    return <h2>User ID: {id}</h2>;
7  }
8
9  function App() {
10   return (
11     <BrowserRouter>
12       <Routes>
13         <Route path="/user/:id" element={<User />} />
14       </Routes>
15     </BrowserRouter>
16   );
17 }
18
19 export default App;
20
```

3. Define the Context API and its role in global state management?

Context is a mechanism that provides different Components and allows us to pass data without doing prop drilling.

2.1 Creating Context

Syntax:

```
1 React.createContext(INITIAL_VALUE)
```

It returns an object with different properties to update and access values from the context.

2.2 Context Properties

The Context object provides two properties.

Consumer

Provider

2.3 How to Access the Value?

We can access the value in the Context using **Consumer Component** provided by the Context Object.

4. What is Client-Server Communication in the context of a React app?

Answer: It refers to the process where a React application (the Client) sends requests to a backend API or web server (the Server) to fetch, create, update, or delete data.

Mechanism: This is typically handled using the `fetch()` API or libraries like `Axios`, usually triggered within a `useEffect` hook to ensure data is synchronized with the UI.

5. What are Nested Routes, and when should they be used?

Nested Routes are routes inside other routes.

🔗 In a React app (using React Router), they allow you to **render a component inside another component based on the URL**.

Why Use Nested Routes?

Because many apps have **layouts inside layouts**.

Example:

- Dashboard page with sidebar
- Inside it → different sections (profile, settings, analytics)

🔗 You don't want to reload the whole page — only the inner content changes.

1. Define Routes

```
</> JavaScript
import { BrowserRouter, Routes, Route } from "react-router-dom";

<BrowserRouter>
  <Routes>
    <Route path="/dashboard" element={<Dashboard />}>
      <Route path="profile" element={<Profile />} />
      <Route path="settings" element={<Settings />} />
    </Route>
  </Routes>
</BrowserRouter>
```

LONG ANSWERS

1. Compare the Context API with the Reducer Pattern for managing advanced application state?

Aspect	Context API	Reducer Pattern (useReducer)
Definition	A feature of React that allows sharing data globally without prop drilling	A state management pattern in React using a reducer function to handle state updates
Purpose	To provide global access to data across components	To manage complex state logic in a predictable way
Working Principle	Uses Provider to supply data and useContext to consume it	Uses state, dispatch, and reducer(state, action) to update state
State Management	Basic / simple state sharing	Advanced / structured state handling
Data Flow	Direct access from context (no strict flow)	Unidirectional flow (action → reducer → new state)
Code Complexity	Simple and easy to implement	Slightly complex but more organized
Performance	May cause unnecessary re-renders when value changes	Better control over updates; can optimize logic
Usage Example	Theme, authentication, language settings	Shopping cart, form handling, dashboards
Combination Usage	Used to provide global access to state	Used inside Context for managing state logic
Conclusion	Best for sharing global data in small-medium apps	Best for handling complex state logic; often used with Context

2. Explain the implementation of Application Routing, including navigation components like Link and NavLink?

◆ What is Application Routing?

Application Routing is the process of navigating between different pages (components) in a React app **without reloading the page**.

👉 It is handled using React Router.

◆ Why Routing is Needed?

In a normal website:

- Each page → separate HTML file

In a React app:

- Single Page Application (SPA)
 - Routing dynamically changes components based on URL
-

◆ Core Components of React Router

1. **BrowserRouter** → Wraps the app
2. **Routes** → Container for routes
3. **Route** → Defines path + component

◆ Basic Implementation

```
JavaScript
import { BrowserRouter, Routes, Route } from "react-router-dom";

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}
```

◆ Navigation Components

◆ 1. Link

👉 Used to navigate between pages **without refreshing**

```
JavaScript
import { Link } from "react-router-dom";

<Link to="/about">Go to About</Link>
```

✓ Works like `<a>` tag but no page reload

◆ 2. NavLink

👉 Special version of Link that knows **which route is active**

```
JavaScript
import { NavLink } from "react-router-dom";

<NavLink to="/about">About</NavLink>
```

3. Describe the general workflow for implementing user authentication in react application?

User authentication in a React application ensures that only authorized users can access protected parts of the app. It involves coordination between the **frontend (React)**, **backend (e.g., Node.js + Express.js)**, and secure storage using **cookies with JWT tokens**.

◆ 1. User Input and Credentials Submission

- Create **Login/Signup forms** using React components
- Capture user credentials (email, password)
- Send data to backend using `fetch()`

2. Backend Verification and Token Issuance

- Server validates credentials from database
- If valid → generates **JWT (JSON Web Token)**
- Stores JWT in **HTTP-only cookie**

3. Client-Side Token Storage (Cookies)

- JWT is stored automatically in browser cookies
- No manual handling required in React
- HTTP-only cookies improve security (prevent XSS attacks)

4. Global State Management

Create Auth Context: Use the React Context API to manage a global `isAuthenticated` state and user profile data.

Custom Hooks: Implement a `useAuth` hook to easily access authentication functions (login, logout) and the current user's status throughout the app.

5. Routing and UI Protection

Protected Routes: Use React Router to create a wrapper component (e.g., `ProtectedRoute`) that checks if a user is authenticated before rendering a page.

Redirects: Automatically redirect unauthenticated users to the login page when they attempt to access private routes.

Conditional Rendering: Dynamically show or hide UI elements (like "Profile" links or "Logout" buttons) based on the authentication status.

6. Authenticated API Requests

Authorization Header: For subsequent requests to protected API endpoints, attach the stored token in the Authorization header as a Bearer <token> .

Intercept Responses: Use Axios Interceptors to handle common authentication errors, such as 401 Unauthorized.

7. Logout Flow

Clear Session: Remove the token from Cookies storage and reset the global auth state.

Redirect: Send the user back to the login or home page after the session is cleared.

4. Discuss Advanced state management using the context API to solve the prop drilling problem?

1. The Prop Drilling Problem

Prop drilling occurs when data is passed from a parent component down to deeply nested child components through multiple intermediate layers — even when those intermediate components don't need the data themselves.

Example Structure:

```
App
├── Parent
│   └── Child
│       └── GrandChild ← needs data from App
```

If GrandChild needs data from App, it must travel through Parent and Child, even though they don't use it.

Problems with Prop Drilling:

- **Code Clutter** — Intermediate components carry unnecessary props, making code harder to read
- **Maintenance Issues** — Changing props in a deeply nested component requires updating every component in the chain

- **Reduced Reusability** — Components become tightly coupled to specific props, limiting reusability
- **Tight Coupling** — Components depend on each other unnecessarily

2. What is the Context API?

The **React Context API** is a built-in React feature that provides a **direct channel** to share data globally across the component tree — without manually passing props through every intermediate component.

3. Core Mechanism — 3 Steps

✓ Step 1: Create a Context

Use `React.createContext()` to create a context object with a Provider and Consumer.

```
javascript
import React, { createContext } from 'react';

const UserContext = createContext(defaultValue);
```

✓ Step 2: Provide the Context

Wrap the components that need access to the data with the **Provider** component and pass state as a value prop.

```
javascript
function App() {
  const [user, setUser] = useState({ name: 'John Doe' });

  return (
    <UserContext.Provider value={{ user, setUser }}>
      <MainContent />
    </UserContext.Provider>
  );
}
```

All components **within the Provider's scope** can access this value directly.

✓ Step 3: Consume the Context

In **any descendant component**, use the `useContext` hook to access the provided value directly — regardless of nesting depth.

```

javascript

import { useContext } from 'react';

function Profile() {
  const { user } = useContext(UserContext);
  return <p>Welcome, {user.name}</p>;
}

```

No need to pass props through Parent or Child — Profile accesses data **directly**.

4. Comparison: Prop Drilling vs Context API

Feature	Prop Drilling	Context API
Data Flow	Through every layer	Direct access
Code Clarity	Cluttered & complex	Clean & readable
Maintenance	Difficult	Easy
Reusability	Limited	High
External Library	Not needed	Not needed
Scalability	Poor	High

5. Benefits of Context API over Prop Drilling

- **Direct Data Access** — Components access global state directly, eliminating unnecessary prop passing through intermediate components
- **Cleaner Codebase** — Reduces unnecessary props in the component tree, leading to more maintainable code
- **Built-in Solution** — No external libraries required for basic global state management
- **Improved Scalability** — Manages application-wide data in a centralized manner, making the app more scalable as it grows

5. Explain the Route Setup and Implementation process in React Router, including browser Router, Routes and Route?

1. What is React Router?

React Router is a standard library for routing in React applications. It enables **navigation between different views/pages** without reloading the browser — making it a **Single Page Application (SPA)**.

2. Installation

```
bash  
npm install react-router-dom
```

3. Core Components

Component	Purpose
BrowserRouter	Wraps the entire app, enables routing
Routes	Container that holds all Route definitions
Route	Maps a URL path to a specific component
Link	Navigation without page reload
NavLink	Link with active styling support

4. BrowserRouter

What is it?

- **BrowserRouter** is the **top-level wrapper** component
- It uses the **HTML5 History API** to keep UI in sync with the URL
- Must wrap the **entire application** to enable routing

```
</> JavaScript  
  
import { BrowserRouter } from "react-router-dom";  
  
<BrowserRouter>  
  <App />  
</BrowserRouter>
```

◆ 2. Routes

- It acts as a **container** for all routes
- It checks the URL and renders the matching route

```
</> JavaScript  
  
import { Routes } from "react-router-dom";  
  
<Routes>  
  { /* routes go here */ }  
</Routes>
```

◆ 3. Route

- Defines a **path** and the component to display

JavaScript

```
import { Route } from "react-router-dom";

<Route path="/about" element={<About />} />
```

👉 When URL = /about, the About component is shown.

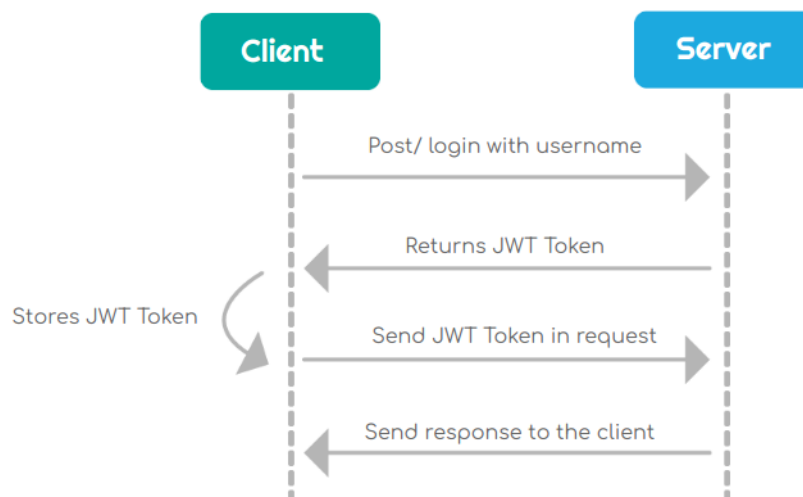
◆ Complete Example

JavaScript

```
import { BrowserRouter, Routes, Route } from "react-router-dom";

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}
```

6. Explain the workflow of Client-Server Communication using fetch or Axios within a component's lifecycle?



◆ 1. Login Request from Client

- The client (browser/app) sends a **POST request** using fetch() with username/password.

```
</> JavaScript
fetch("/login", {
  method: "POST",
  body: JSON.stringify({ username, password })
})
```



◆ 2. Server Validates & Sends JWT

- The server checks the credentials.
 - If valid, it returns a **JWT (JSON Web Token)** as a response.
-

◆ 3. Client Stores JWT

- The client stores the token (in **localStorage** / **sessionStorage**).
 - This token is used for future requests.
-

◆ 4. Sending JWT in Requests

- For every protected request, the client sends the JWT in headers:

```
</> JavaScript
fetch("/data", {
  headers: {
    Authorization: "Bearer <JWT_TOKEN>"
  }
})
```



◆ 5. Server Verifies Token

- The server checks if the JWT is valid.
 - If valid → processes request
 - If invalid → rejects request
-

◆ 6. Server Sends Response

- After verification, the server sends the requested data back to the client.
-

◆ 7. Client Updates UI

- The client receives the response and updates the UI (display data).